AMOSKYS

ADAPTIVE MULTI-AGENT RELIABILITY DRIFT RECALIBRATION MECHANISM

Technical Mechanism Specification v0.1
February 2026

Classification	CONFIDENTIAL — Internal Use Only
Document Owner	Akash Thanneeru — Founder & CEO
Architecture Core	Akash Thanneeru + CTO (Confidential)
Technical Reviewer	Claude Opus 4.6
Status	DESIGN — Pre-Implementation
Depends On	Architecture Foundation Document v1.0

TABLE OF CONTENTS

1. Executive Summary

This document specifies the Adaptive Multi-Agent Reliability Drift Recalibration (AMRDR) mechanism for the AMOSKYS security platform. AMRDR continuously models the statistical reliability of each detection agent as a probability distribution, detects when that reliability drifts from established baselines, and dynamically recalibrates the weight assigned to each agent's outputs in the fusion layer.

The mechanism addresses a structural gap in current enterprise security architectures: while platforms monitor agent health as a binary state (online/offline) or coarse metric (CPU, memory), none model agent detection reliability as a continuous probabilistic variable that influences fusion decisions. This means a degraded agent's outputs carry the same weight as a healthy agent's outputs, directly contributing to false positives and missed detections.

AMRDR is not a scoring algorithm. It is a meta-layer that governs how much the system trusts each agent's scores. It operates above the detection layer and below the incident layer, modifying the fusion process itself.

> **Core Claim**
>
> A method for continuously estimating per-agent detection reliability using Bayesian posterior distributions, detecting reliability drift through windowed statistical tests, and dynamically recalibrating fusion weights in a distributed multi-agent threat detection system, such that the composite threat assessment reflects the measured trustworthiness of each contributing agent at the time of fusion.

2. Problem Definition

2.1 The Static Weight Problem

In AMOSKYS's current architecture, the Fusion Engine evaluates events from 10+ agents using correlation rules with fixed thresholds. Each correlation rule assigns fixed severity when conditions match. The ScoreJunction computes composite ThreatScores (0–100) using rule-matched correlation with static score_weight values per rule. These weights do not change based on whether the contributing agent is producing accurate detections.

Concretely, in the current codebase:

- ScoreJunction correlation rules have fixed score_weight fields (0.5, 0.6, 0.7) that never change.
- Fusion Engine risk contributions are hardcoded: failed SSH = +5, new LaunchAgent = +25, CRITICAL incident = +40.
- ThreatScore confidence is derived from event count (min(1.0, event_count / 10)), not from agent reliability.
- DeviceRiskSnapshot decay is a fixed -10 per 10-minute period, independent of agent accuracy.

2.2 Why Agents Drift

Agent detection reliability degrades over time for specific, measurable reasons:

- **Environmental Change:** OS updates modify system call behavior, invalidating process behavior baselines. Application updates change normal network patterns. User behavior shifts with organizational changes.
- **Sensor Drift:** File integrity baselines become stale as legitimate updates accumulate. DNS query patterns shift with infrastructure changes. Authentication patterns change with employee turnover.
- **Adversarial Manipulation:** Attackers may suppress or forge telemetry to degrade specific agent accuracy. AMOSKYS already detects some evasion techniques (HardenedAgentBase.detect_evasion_attempts), but this defense is binary, not probabilistic.
- **Baseline Staleness:** The kernel audit agent's behavioral model was trained on a distribution that has since shifted. The persistence guard's list of known-good launch agents becomes incomplete.

2.3 Consequences of Ignoring Drift

When an agent drifts and its outputs retain full weight in fusion:

- False positive rate increases as a degraded agent produces spurious alerts at the same authority level.
- False negative rate increases as a compromised agent's missed detections are not compensated by reduced trust.

- Composite ThreatScores become unreliable because they incorporate unreliable inputs at face value.
- Correlation rules fire on noisy data, creating incidents that waste analyst investigation time.
- DeviceRiskSnapshots accumulate points from degraded agents, producing inflated risk scores.

3. Mathematical Foundation

3.1 Agent Reliability as a Probability Distribution

We define agent reliability not as a single number but as a probability distribution over the agent's true detection accuracy. This captures both the current estimate and the uncertainty in that estimate.

3.1.1 The Beta-Binomial Model

For each agent i, we model its reliability θ_i as a random variable following a Beta distribution:

$$\theta_i \sim Beta(\alpha_i, \beta_i)$$

$$\text{where } \alpha_i \text{ encodes observed correct detections}$$
$$\text{and } \beta_i \text{ encodes observed incorrect detections}$$

The Beta distribution is the conjugate prior for Bernoulli-distributed observations, which means posterior updates have a closed-form solution. When we observe a correct detection (true positive or true negative), we increment α. When we observe an incorrect detection (false positive or false negative), we increment β.

3.1.2 Posterior Update Rule

Given a new batch of observations where the agent produced H correct and T incorrect assessments:

$$\text{Posterior: } Beta(\alpha_i + H, \beta_i + T)$$

$$\text{Point estimate (posterior mean): } \hat{\rho}_i = (\alpha_i + H) / (\alpha_i + H + \beta_i + T)$$

$$\text{Posterior variance: } Var(\theta_i) = (\alpha_i\beta_i) / ((\alpha_i+\beta_i)^2(\alpha_i+\beta_i+1))$$

The posterior mean $\hat{\rho}_i$ is the system's best estimate of agent i's current reliability. The posterior variance quantifies uncertainty in that estimate. A new agent with few observations will have high variance (low confidence in the reliability estimate). A well-observed agent will have low variance (high confidence).

3.1.3 Prior Selection

The choice of initial prior (α_0, β_0) reflects our belief about a new agent before any observations:

Prior	Interpretation	Use Case
Beta(1, 1)	Uniform (no prior belief)	New agent, no history
Beta(10, 2)	Strong prior: ~83% reliable	Validated agent type
Beta(5, 5)	Uncertain: centered at 50%	Untested agent in new env

For AMOSKYS, we recommend Beta(10, 2) as the default prior for validated agent types (Process Monitor, Auth Guard, etc.), reflecting the expectation that a properly configured agent starts with high reliability. New or experimental agents should use Beta(2, 2), a weakly informative prior centered at 50% with low confidence.

3.1.4 Credible Intervals for Reliability

The 95% Highest Posterior Density (HPD) interval provides actionable bounds on an agent's reliability:

$$HPD_{95} = [Beta.ppf(0.025, \alpha_i, \beta_i), Beta.ppf(0.975, \alpha_i, \beta_i)]$$

If the lower bound of the HPD interval drops below a configured threshold (e.g., 0.5), this constitutes a reliability alarm: the system has statistical evidence that the agent may be operating below acceptable accuracy.

3.2 Drift Detection

Drift detection determines when an agent's reliability has changed significantly from its established baseline. We employ a dual-detector architecture combining a fast detector (ADWIN) for abrupt drift and a sensitive detector (EDDM) for gradual drift.

3.2.1 ADWIN: Adaptive Windowing for Abrupt Drift

ADWIN maintains a variable-length sliding window of agent error observations and tests whether any two contiguous sub-windows have statistically different means.

For each incoming observation x_t (where 1 = error, 0 = correct):

$$\text{Append } x_t \text{ to window } W$$
$$\text{For each split point dividing } W \text{ into } W_0 \text{ and } W_1\text{:}$$
$$\text{Test: } |\text{mean}(W_0) - \text{mean}(W_1)| > \varepsilon_t$$

$$\text{where } \varepsilon_t = \sqrt{((1/2n_0n_1) \cdot \ln(2/\delta))}$$

$$\delta = \text{significance threshold (default: } 0.002)$$
$$n_0 = |W_0|,\ n_1 = |W_1|$$

When drift is detected, ADWIN drops the oldest sub-window and resets. The key property is that window size adjusts automatically: large windows during stable periods (more statistical power) and small windows after change (fast adaptation).

3.2.2 EDDM: Early Drift Detection for Gradual Degradation

EDDM tracks the distance between consecutive errors rather than error rate, making it sensitive to gradual reliability degradation where errors spread apart before eventually clustering.

$$d_i = \text{distance from error } i \text{ to error } i\text{-}1 \text{ (in events)}$$

$$\text{Track: mean distance } \bar{d}_t \text{ and standard deviation } s_t$$

$$\text{Warning: } \bar{d}_t + s_t < (\bar{d}_min + s_min) \cdot 0.95$$
$$\text{Drift: } \quad \bar{d}_t + s_t < (\bar{d}_min + s_min) \cdot 0.90$$

EDDM fires a warning before declaring drift, allowing the system to enter a heightened monitoring state before triggering recalibration.

3.2.3 Combined Detection Logic

The dual-detector architecture uses the following decision table:

ADWIN State	EDDM State	System State	Action
No drift	No drift	STABLE	Normal operation
No drift	Warning	MONITORING	Increase observation freq
No drift	Drift	GRADUAL_DRIFT	Soft recalibrate

Drift	Any	ABRUPT_DRIFT	Hard recalibrate
Drift	Drift	CRITICAL_DRIFT	Quarantine + alert

3.3 Dynamic Weight Recalibration

3.3.1 Reliability-Weighted Fusion

The core mechanism replaces static weights in the fusion layer with dynamic reliability-derived weights. For K agents contributing to a composite threat assessment:

$$Threat_fused(t) = \Sigma_i \hat{\rho}_i(t) \cdot x_i(t) \,/\, \Sigma_i \hat{\rho}_i(t)$$

where:

$\hat{\rho}_i(t)$ = *posterior mean reliability of agent i at time t*

$x_i(t)$ = *detection score from agent i at time t*

This means an agent with reliability 0.95 contributes nearly full weight, while an agent with reliability 0.40 contributes less than half weight. The fusion output naturally reflects the trustworthiness of its inputs.

3.3.2 Soft Recalibration (Gradual Drift)

When gradual drift is detected (EDDM fires, ADWIN does not), the system applies soft recalibration:

Apply exponential forgetting to Beta parameters:

$$\alpha_i' = \alpha_i \cdot \lambda + \alpha_o \cdot (1 - \lambda)$$
$$\beta_i' = \beta_i \cdot \lambda + \beta_o \cdot (1 - \lambda)$$

where λ = *forgetting factor (default: 0.85)*

and (α_o, β_o) = *initial prior*

This gradually pulls the posterior toward the prior, reducing the influence of stale historical observations while retaining some memory. The agent's weight in fusion decreases proportionally as its recent performance diverges from historical performance.

3.3.3 Hard Recalibration (Abrupt Drift)

When abrupt drift is detected (ADWIN fires), the system applies hard recalibration:

Reset Beta parameters to weakened prior:

$$\alpha_i' = \max(2, \alpha_o / 2)$$
$$\beta_i' = \max(2, \beta_o / 2)$$

Begin rapid re-estimation from recent observations only

Hard recalibration effectively says: "We no longer trust the historical reliability estimate. Start over with weak prior beliefs and let recent observations dominate." The agent's weight in fusion drops immediately and recovers only as new evidence demonstrates renewed accuracy.

3.3.4 Quarantine (Critical Drift)

When both detectors fire simultaneously, the system quarantines the agent:

- Agent's weight in fusion is set to 0 (complete exclusion from composite scoring).

- Agent continues operating and collecting telemetry (data is preserved for forensic analysis).
- A ReliabilityIncident is generated and surfaced in the Intelligence dashboard.
- Agent must pass a recovery verification window (configurable, default: 50 observations with >80% accuracy) before weight is restored.

4. Architecture Integration

AMRDR integrates into the existing AMOSKYS architecture as a new component between the EventBus and the Fusion Engine, without modifying the agent layer or the transport protocol.

4.1 Component Placement

Existing Layer	AMRDR Component	Integration Point
Agents	No change	Agents remain unaware of AMRDR
EventBus	Observation Collector	New subscriber to telemetry stream
ScoreJunction	Weight Injector	Modifies score_weight per agent
Fusion Engine	Risk Modifier	Scales risk contributions by reliability
Dashboard	Reliability Display	New visualization panel
Storage	Reliability Store	New SQLite table for posteriors

4.2 Data Flow

The AMRDR data flow operates as follows:

1. Agents publish TelemetryEvents through the EventBus (unchanged).
2. The Observation Collector subscribes to the EventBus and extracts per-agent observation data (events + their eventual validation status).
3. For each agent, the Reliability Estimator updates the Beta posterior with new observations.
4. The Drift Detector runs ADWIN and EDDM on the error stream for each agent.
5. If drift is detected, the Recalibrator applies soft/hard/quarantine recalibration to the affected agent's posterior.
6. The Weight Injector computes current reliability weights and injects them into the ScoreJunction's score_weight fields and the Fusion Engine's risk contribution multipliers.
7. Composite ThreatScores and DeviceRiskSnapshots now reflect per-agent trustworthiness.

4.3 Existing Extension Points

AMRDR leverages the following existing AMOSKYS infrastructure without modification:

- **TelemetryEvent.confidence_score (0.0–1.0):** Already defined in universal_telemetry.proto. AMRDR reads this field but does not write it — the agent's self-reported confidence is an input, not an output.
- **SecurityContext.device_trust_score (0.0–1.0):** Already defined in proto. AMRDR can write this field with the Bayesian posterior mean, providing downstream consumers with a continuously calibrated trust score.
- **SecurityContext.anomaly_score (0.0–1.0):** Available for AMRDR to populate with drift severity metrics.

- **CircuitBreaker pattern:** The existing circuit breaker in HardenedAgentBase provides binary failure detection. AMRDR extends this with probabilistic degradation detection — the circuit breaker catches crashes, AMRDR catches accuracy drift.
- **Collection statistics:** HardenedAgentBase already tracks total_events, events_per_collection, collection_durations_ms, gaps_detected, and anomalies_detected. These feed directly into AMRDR's observation stream.

5. Ground Truth Acquisition

The Bayesian update mechanism requires classifying each agent observation as correct or incorrect. In a security context, ground truth is inherently delayed and imperfect. This section defines how AMRDR acquires ground truth signals without requiring perfect labels.

5.1 Ground Truth Sources

5.1.1 Cross-Agent Consensus

When multiple independent agents observe the same entity (e.g., a process both spawns a suspicious child and makes an anomalous network connection), agreement between agents constitutes a weak ground truth signal:

- If 3+ agents flag the same entity within a time window, treat each agent's assessment as likely correct.
- If one agent flags an entity that 3+ other agents assess as benign, treat the outlier as likely incorrect.

This is not perfect ground truth but provides a continuous, automated signal. The strength of the signal increases with the number of agreeing agents.

5.1.2 Incident Confirmation

When the Fusion Engine creates an Incident, the contributing events receive implicit positive validation. When an analyst confirms or rejects an incident (planned enhancement), this provides strong ground truth:

- Confirmed incident: All contributing agent observations marked correct (H += 1).
- Rejected incident (false positive): Contributing agent observations marked incorrect (T += 1).
- Analyst feedback weight is configurable (default: 5x automatic signals) to reflect the higher confidence of human validation.

5.1.3 Temporal Decay Validation

Events that do not escalate into incidents within the Fusion Engine's time window receive a weak negative signal for HIGH/CRITICAL severity events (if an agent reports critical severity but no incident materializes after correlation, this is evidence of potential over-reporting) and a weak positive signal for INFO/LOW severity events (the agent correctly did not over-escalate).

5.1.4 Integrity Verification

HardenedAgentBase already performs periodic integrity checks. When verify_integrity() detects tampering (source code changed, config modified, suspicious environment variables), this is treated as a strong negative signal:

- Agent compromised indicator: T += 10 (equivalent to 10 incorrect observations).
- This dramatically shifts the posterior toward low reliability and triggers hard recalibration.

5.2 Observation Weighting

Not all ground truth sources carry equal evidential weight:

Source	Weight	Rationale
Analyst confirmation	5.0	Highest confidence: human expert judgment
Integrity failure	10.0 (negative only)	Direct evidence of compromise
Cross-agent consensus (3+)	1.0	Independent agreement, standard weight
Cross-agent disagreement	0.5	One agent vs. others, lower confidence
Incident contribution	1.0	Incident creation implies corroborated detection
Temporal non-escalation	0.3	Weakest signal: absence of incident is not proof of correctness

6. Core Algorithm

6.1 Per-Agent Reliability Estimator

The following pseudocode describes the complete per-agent reliability estimation process:

```
CLASS AgentReliabilityEstimator:
  INIT(agent_id, prior_alpha=10, prior_beta=2, adwin_delta=0.002):
    self.agent_id = agent_id
    self.alpha = prior_alpha
    self.beta = prior_beta
    self.alpha_0 = prior_alpha          // Original prior for resets
    self.beta_0 = prior_beta
    self.adwin = ADWIN(delta=adwin_delta)
    self.eddm = EDDM()
    self.state = STABLE
    self.observation_count = 0
    self.quarantine_recovery_count = 0

  PROPERTY reliability -> float:
    RETURN self.alpha / (self.alpha + self.beta)

  PROPERTY uncertainty -> float:
    n = self.alpha + self.beta
    RETURN sqrt((self.alpha * self.beta) / (n * n * (n + 1)))

  PROPERTY hpd_95 -> (float, float):
    RETURN (Beta.ppf(0.025, self.alpha, self.beta),
            Beta.ppf(0.975, self.alpha, self.beta))

  METHOD observe(is_correct: bool, weight: float = 1.0):
    IF is_correct:
      self.alpha += weight
    ELSE:
      self.beta += weight
    self.observation_count += 1

    // Feed error stream to drift detectors
    error = 0 IF is_correct ELSE 1
    adwin_drift = self.adwin.add_element(error)
    eddm_state = self.eddm.add_element(error)

    // Determine system state from dual detectors
    self._update_state(adwin_drift, eddm_state)

  METHOD _update_state(adwin_drift, eddm_state):
    IF adwin_drift AND eddm_state == DRIFT:
      self.state = CRITICAL_DRIFT
      self._quarantine()
    ELIF adwin_drift:
      self.state = ABRUPT_DRIFT
      self._hard_recalibrate()
    ELIF eddm_state == DRIFT:
      self.state = GRADUAL_DRIFT
      self._soft_recalibrate(lambda=0.85)
    ELIF eddm_state == WARNING:
      self.state = MONITORING
    ELSE:
      IF self.state == QUARANTINE:
```

```
          self._check_recovery()
        ELSE:
          self.state = STABLE
```

6.2 Recalibration Methods

```
    METHOD _soft_recalibrate(lambda=0.85):
      // Exponential forgetting: blend toward prior
      self.alpha = self.alpha * lambda + self.alpha_0 * (1 - lambda)
      self.beta  = self.beta  * lambda + self.beta_0  * (1 - lambda)
      EMIT RecalibrationEvent(agent_id, type=SOFT,
  new_reliability=self.reliability)

    METHOD _hard_recalibrate():
      // Reset to weakened prior
      self.alpha = max(2, self.alpha_0 / 2)
      self.beta  = max(2, self.beta_0  / 2)
      self.adwin.reset()
      EMIT RecalibrationEvent(agent_id, type=HARD,
  new_reliability=self.reliability)

    METHOD _quarantine():
      self.state = QUARANTINE
      self.quarantine_recovery_count = 0
      EMIT ReliabilityIncident(agent_id, severity=HIGH,
        summary='Agent quarantined: both drift detectors triggered')

    METHOD _check_recovery():
      // Agent must demonstrate accuracy to exit quarantine
      IF last_observation_was_correct:
        self.quarantine_recovery_count += 1
      ELSE:
        self.quarantine_recovery_count = max(0, self.quarantine_recovery_count - 2)
      IF self.quarantine_recovery_count >= RECOVERY_THRESHOLD:  // default: 50
        self.state = STABLE
        self._hard_recalibrate()  // Start with clean posterior
        EMIT RecoveryEvent(agent_id)
```

6.3 Fusion Weight Injection

The Weight Injector modifies fusion behavior at two integration points:

6.3.1 ScoreJunction Integration

```
// Current (static):
rule.score_weight = 0.7  // Fixed

// With AMRDR:
effective_weight = rule.score_weight * agent_reliability(source_agent_id)

// Example: if process_monitor has reliability 0.60:
// effective_weight = 0.7 * 0.60 = 0.42  (reduced from 0.7)
```

6.3.2 Fusion Engine Integration

```
// Current (static):
risk_contribution = {
  'failed_ssh': 5,
  'new_launch_agent': 25,
  'suspicious_sudo': 30,
  'critical_incident': 40
}

// With AMRDR:
source_reliability = agent_reliability(event.collection_agent)
risk_contribution = base_contribution * source_reliability

// Example: if auth_guard has reliability 0.45:
// failed_ssh contribution = 5 * 0.45 = 2.25  (reduced from 5)
// suspicious_sudo contribution = 30 * 0.45 = 13.5  (reduced from 30)
```

6.3.3 Quarantine Exclusion

```
// When agent is quarantined:
IF agent_state(source_agent_id) == QUARANTINE:
  effective_weight = 0.0      // Complete exclusion from scoring
  risk_contribution = 0       // No risk accumulation
  // Events still stored for forensic analysis
  // Agent telemetry still logged to WAL
```

7. Configuration Parameters

Parameter	Default	Description
`prior_alpha`	10	Initial Beta alpha (correct observations prior)
`prior_beta`	2	Initial Beta beta (incorrect observations prior)
`adwin_delta`	0.002	ADWIN significance threshold (lower = less sensitive)
`eddm_warning_threshold`	0.95	EDDM warning level multiplier
`eddm_drift_threshold`	0.90	EDDM drift level multiplier
`soft_recal_lambda`	0.85	Forgetting factor for soft recalibration
`recovery_threshold`	50	Correct observations to exit quarantine
`recovery_accuracy`	0.80	Min accuracy during recovery window
`min_observations`	30	Minimum observations before drift detection activates
`reliability_alarm_floor`	0.50	HPD lower bound below which alarm fires
`analyst_feedback_weight`	5.0	Multiplier for analyst-confirmed observations
`consensus_min_agents`	3	Minimum agents for cross-agent consensus

8. Failure Modes and Mitigations

8.1 Enumerated Failure Modes

Failure Mode	Cause	Impact	Mitigation
Poisoned ground truth	Attacker manipulates cross-agent consensus	Reliable agents downweighted	Byzantine-tolerant consensus (>1/3 honest agents)
Posterior explosion	Alpha/beta grow unbounded over time	System becomes unresponsive to drift	Periodic normalization: cap alpha+beta at 1000
Cascading quarantine	Correlated environmental change triggers multi-agent drift	Too many agents quarantined simultaneously	Quarantine limit: max 30% of agents. Excess get soft recal only
Cold start inaccuracy	Insufficient observations for posterior to stabilize	Premature drift detection	min_observations guard (default: 30) before drift detection
Label delay	Ground truth arrives long after detection	Stale posterior updates	Observation timestamp tracking. Reject labels >1h stale
AMRDR component failure	Reliability estimator crashes	No weight adjustment, revert to static	Graceful fallback: if AMRDR unavailable, use weight=1.0 for all agents

8.2 Graceful Degradation

AMRDR is designed as an enhancement layer, not a dependency. If the AMRDR component fails:

- All agent weights revert to 1.0 (equivalent to current static behavior).
- The Fusion Engine and ScoreJunction continue operating with their original fixed weights.
- A system health alert is generated for the AMRDR component itself.
- No telemetry data is lost; agents and EventBus are unaffected.

This ensures AMRDR adds value when operational but never causes system-wide failure when unavailable.

9. Statistical Basis and Theoretical Guarantees

9.1 Beta-Binomial Conjugacy

The use of Beta distributions for reliability estimation is not arbitrary. The Beta-Binomial conjugate pair provides:

- **Closed-form updates:** No iterative optimization or sampling required. Posterior computation is O(1) per observation.
- **Natural uncertainty quantification:** Posterior variance decreases monotonically with observation count, providing automatic confidence tracking.
- **Interpretable parameters:** Alpha and beta have direct physical meaning: counts of correct and incorrect observations, respectively.
- **Numerical stability:** Beta distribution parameters are bounded and well-conditioned for all valid inputs.

9.2 ADWIN Guarantees

ADWIN (Bifet and Gavalda, 2007) provides the following mathematical guarantees:

- False positive rate bounded by delta (probability of detecting drift when none exists).
- Detection delay bounded by O(1/epsilon^2 * log(1/delta)) where epsilon is the magnitude of change.
- Memory usage bounded by O(1/epsilon * log(n/delta)) where n is the window size.
- Window size adapts automatically: large during stable periods, small after detected change.

9.3 Convergence Properties

The reliability estimate converges to the true agent reliability under the following conditions:

- Agent reliability is approximately stationary within drift detection windows (piecewise-stationary assumption).
- Ground truth observations are conditionally independent given the agent's true reliability.
- The observation weighting scheme does not introduce systematic bias.

Under these conditions, the posterior mean converges to the true reliability at rate O(1/sqrt(n)) where n is the number of observations, and the HPD interval contracts at the same rate.

9.4 Calibration Verification

AMRDR monitors its own calibration using Expected Calibration Error (ECE):

> $$ECE = \Sigma_m\ (|B_m|/n) \cdot |acc(B_m) - conf(B_m)|$$
>
> *where B_m are confidence bins, acc is observed accuracy, and conf is predicted reliability*

If ECE exceeds 0.10 (10% calibration error), the system applies temperature scaling to the reliability outputs and generates a self-diagnostic alert. This ensures AMRDR does not degrade the system's overall accuracy through miscalibrated reliability estimates.

10. Prior Art Differentiation

This section precisely delineates how AMRDR differs from existing patented approaches and academic systems.

Dimension	Prior Art	AMRDR	Structural Difference
Reliability model	Static trust scores or binary health	Continuous Beta posteriors per agent	Probabilistic vs. point estimate
Drift detection	Manual threshold tuning	Automatic ADWIN + EDDM dual detectors	Guaranteed bounds vs. ad-hoc thresholds
Recalibration	Manual retraining or restart	Three-tier automated recalibration	Autonomous vs. operator-dependent
Weight dynamics	Pre-computed static	Real-time reliability-weighted	Dynamic vs. fixed
Uncertainty	Not exposed	HPD intervals per agent	Quantified vs. absent
Ground truth	Requires labeled data	Multi-source: consensus + incidents + analyst + integrity	Continuous vs. batch-labeled
Self-monitoring	None	ECE calibration verification	Self-aware vs. blind
Failure handling	Binary: working/broken	Graceful: 5 states from STABLE to QUARANTINE	Graduated vs. binary

10.1 Closest Existing Patent

US Patent 12,063,200B2 (Systems and Methods for Sensor Trustworthiness) evaluates cyberthreat sensor data against expected ranges and assesses sensor health. AMRDR differs in three structural ways: (1) trustworthiness is modeled as a probability distribution, not a score; (2) drift detection uses mathematically-guaranteed algorithms (ADWIN), not range comparison; and (3) recalibration directly modifies fusion weights rather than generating advisory signals.

11. Implementation Roadmap

11.1 Phase 1: Core Estimator (Weeks 3–4)

- Implement AgentReliabilityEstimator class with Beta posterior, ADWIN, EDDM.
- Unit tests: posterior update correctness, drift detection sensitivity, recalibration behavior.
- Synthetic benchmark: simulated agent with known reliability transitions.
- No integration with live system yet.

11.2 Phase 2: Ground Truth Pipeline (Weeks 5–6)

- Implement cross-agent consensus observer on EventBus subscriber.
- Implement incident contribution tracking in Fusion Engine.
- Implement integrity verification signal from HardenedAgentBase.
- Integration test: end-to-end observation flow from agent event to reliability update.

11.3 Phase 3: Fusion Integration (Weeks 7–8)

- Implement Weight Injector for ScoreJunction score_weight modification.
- Implement Risk Modifier for Fusion Engine contribution scaling.
- Implement quarantine exclusion logic.
- Regression test: existing correlation rules produce identical results when all agents have reliability 1.0.

11.4 Phase 4: Dashboard and Monitoring (Weeks 9–10)

- Add reliability visualization panel to Intelligence dashboard.
- Implement ReliabilityIncident and RecalibrationEvent display.
- Add Prometheus metrics for per-agent reliability, drift events, recalibration count.
- Implement ECE self-monitoring with alerting.

11.5 Phase 5: Validation (Weeks 11–12)

- Deploy on test endpoints with simulated agent degradation scenarios.
- Measure: false positive reduction with AMRDR active vs. inactive.
- Measure: detection latency impact (AMRDR computational overhead).
- Validate graceful degradation: kill AMRDR component and verify fallback.

12. Open Questions

The following questions remain unresolved and require Architecture Core decision before implementation:

12.1 Posterior Normalization Strategy

Should alpha + beta be capped at a fixed maximum (e.g., 1000) to prevent the posterior from becoming so concentrated that it resists drift detection? Alternatively, should we use a sliding window approach where only the most recent N observations contribute to the posterior? Both have tradeoffs: capping preserves responsiveness but discards information; windowing preserves recency but increases computational cost.

12.2 Multi-Agent Correlation in Ground Truth

Cross-agent consensus provides ground truth, but it assumes agent independence. In practice, agents monitoring the same device may share environmental biases. Should we model agent correlation explicitly (e.g., via a correlation matrix) and discount consensus from correlated agents? This adds accuracy but significant complexity.

12.3 Adversarial Robustness

A sophisticated attacker who understands AMRDR could deliberately degrade one agent to reduce its weight, then attack through that agent's domain knowing the fusion layer will discount detections from that vector. Should AMRDR include a counter-mechanism that increases monitoring sensitivity on quarantined agent domains rather than simply excluding them?

12.4 Analyst Feedback Priority

The current design weights analyst feedback at 5x automatic signals. Should this weight be configurable per analyst (senior analysts have higher weight) or per incident type (feedback on CRITICAL incidents carries more weight than feedback on LOW incidents)?

12.5 Patent Filing Timing

The mechanism is specified at sufficient detail for a provisional patent application. However, implementation validation (Phase 5) would provide empirical evidence supporting the claims. The tradeoff: filing early secures priority date but without validation data; filing after validation provides stronger claims but risks prior art accumulation in a fast-moving field.

Appendix: Key References

- **Bifet & Gavalda (2007):** Learning from time-changing data with adaptive windowing. SIAM International Conference on Data Mining. [ADWIN algorithm]

- **Baena-Garcia et al. (2006):** Early drift detection method. ECML PKDD Workshop on Knowledge Discovery from Data Streams. [EDDM algorithm]
- **Jøsang (2016):** Subjective Logic: A Formalism for Reasoning Under Uncertainty. Springer. [Computational trust framework]
- **US Patent 12,063,200B2:** Systems and Methods for Sensor Trustworthiness. [Closest prior art for cybersecurity sensor reliability]
- **US Patent 11,089,040B2:** Cognitive analysis of security data with signal flow-based graph exploration. [Graph-based threat scoring with toxicity values]
- **Guo et al. (2017):** On Calibration of Modern Neural Networks. ICML. [Temperature scaling, ECE metric]
- **River ML Library:** https://riverml.xyz — Production implementations of ADWIN, EDDM, Page-Hinkley for online drift detection.
- **AMOSKYS Architecture Foundation v1.0:** Internal document. Defines system architecture, agent specifications, fusion mechanisms, and MVP scope.